Pokhara University

Gandaki College of Engineering and Science

Lamachaur, Kaski

Lab 3: Implementation of Software Testing – Test Case Generation, JUnit, Jest, Selenium, and Load Testing

Submitted By:

Sumin GC

Roll no. 40

Submitted To:

Er. Rajendra Bahadur Thapa

Lab Report

Title

Software Testing Using JUnit, Jest, Selenium, and Apache JMeter

Objective

To implement software testing techniques by generating test cases and using JUnit, Jest, Selenium WebDriver, and Apache JMeter to verify the functionality, reliability, and performance of a software application.

Tools Required

- Java Development Kit (JDK)
- Node.js
- JUnit
- Jest
- Selenium WebDriver
- Apache JMeter
- Visual Studio Code, IntelliJ IDEA, or Eclipse

Theory

Software testing is the process of verifying and validating that a software application functions according to its specified requirements. It helps identify defects, improve software quality, and ensure the reliability of the application before deployment.

Different testing tools are designed for specific testing purposes. JUnit is a unit testing framework for Java applications that verifies the correctness of individual program components. Jest is a JavaScript testing framework used to perform unit and integration testing efficiently. Selenium WebDriver automates web browser interactions, enabling functional and regression testing of web applications. Apache JMeter is a performance testing tool used to simulate multiple users and evaluate application performance by measuring response time, throughput, and system stability under varying workloads.

Procedure

1. Prepare the application and configure the required testing environment.
2. Identify the application's functional requirements and generate appropriate test cases.
3. Execute unit tests for Java components using JUnit.
4. Execute JavaScript unit tests using Jest.
5. Automate browser interactions and functional testing using Selenium WebDriver.
6. Configure Apache JMeter to simulate multiple virtual users and perform load testing.
7. Analyze the testing results, identify any failures or performance issues, and verify that the application behaves as expected.

Sample Test Cases

- Verify login functionality using valid user credentials.
- Verify that login fails when invalid credentials are entered.
- Validate the application's behavior when required input fields are left empty.
- Verify that the logout functionality terminates the user session successfully.
- Confirm that the application produces the expected output for valid inputs.
- Verify that appropriate error messages are displayed for invalid inputs.

Coding Section

JUnit Example

```
import static org.junit.Assert.assertEquals;
import org.junit.Test;

public class CalculatorTest {

   @Test
   public void testAddition() {
      assertEquals(10, 5 + 5);
   }
}
```

Jest Example

```
test("adds two numbers", () => {
   expect(5 + 5).toBe(10);
});
```

Selenium WebDriver Example

```
WebDriver driver = new ChromeDriver();

driver.get("https://example.com");

System.out.println(driver.getTitle());

driver.quit();
```

Load Testing

Apache JMeter was used to create multiple virtual users that generated requests to the application simultaneously. Performance metrics such as response time, throughput, latency, and error rate were monitored and analyzed to evaluate the application's performance under different workload conditions.

Output

- Test cases were successfully designed based on the application requirements.
- JUnit unit tests executed successfully for Java components.
- Jest tests verified the functionality of JavaScript modules.
- Selenium WebDriver automated browser interactions and validated application behavior.
- Apache JMeter simulated concurrent users and measured application performance.
- Functional testing confirmed that the application behaved according to the specified requirements.
- Performance metrics such as response time, throughput, and error rate were successfully analyzed.

Result

The experiment successfully generated and executed software test cases using JUnit, Jest, Selenium WebDriver, and Apache JMeter. The testing process verified the correctness of application functionality, automated browser operations, and evaluated application performance under simulated workloads.

Conclusion

This experiment provided practical experience in implementing different software testing techniques using JUnit, Jest, Selenium WebDriver, and Apache JMeter. Unit testing verified the correctness of individual program components, Selenium automated functional testing of web applications, and

Apache JMeter evaluated system performance under load. Together, these testing tools contribute to developing reliable, high-quality, and efficient software systems.